Deliverable #1: Software Requirement Specification

SE 3A04: Software Design II - Large System Design

Tutorial: T03
Group: G2
Group Members:
McEachern, Connor Joshua
Zafar, Ahmed
Heathfield, Aidan Amato
Patel, Saumy Piyushbhai

[1] Introduction:

This SRS will provide visibility over software requirements for **SCEMAS**, a Smart City Environmental Monitoring & Alert System. This document will discuss the purpose of **SCEMAS**, the scope of the application, user characteristics, product requirements, and use case diagrams.

[1.1] Purpose:

The document focuses on software requirements, user characteristics, and use cases for **SCEMAS**. This document is intended for internal **SCEMAS** stakeholders, including but not limited to project managers, developers, domain experts, and **SCEMAS** team members/investors. No prior readings are required.

[1.2] Scope:

Currently, many cities rely on sparse, manual, or siloed environmental monitoring, leading to delayed responses to pollution events, heatwaves, or noise disturbances. There is a clear need for an integrated, real-time system that can:

- Continuously monitor key environmental indicators (air quality, noise levels, temperature, humidity).
- Automatically detect threshold violations and generate timely alerts.
- Provide a unified operational dashboard for city management.
- Disseminate crucial environmental information to the public to promote awareness and safety.

SCEMAS addresses this need as a cloud-native IoT software platform. The project focuses on the architectural design and core implementation of the system's backend services, data pipelines, and APIs, acknowledging that the physical sensor hardware will be simulated for development and testing purposes.

SCEMAS, the smart city environmental monitoring and alert system, will allow users to view and receive alerts regarding their local environment, and also inform them.

[1.3] Definitions, Acronyms, and Abbreviations

MQTT (Message Queuing Telemetry Transport)
An industry-standard communication protocol to send data to the central system over the internet.

REST-API (Representational State Transfer Application Programming Interface)
An API that allows systems to communicate over the internet using HTTP requests to send and receive data.

SCEMAS (Smart City Environmental Monitoring & Alert System)
A smart city software platform designed to monitor environmental conditions in real time and deliver timely alerts based on collected sensor data.

Telemetry
Time-stamped environmental data (temperature, air quality, noise) collected by sensors and sent to **SCEMAS** for monitoring and analysis.

[1.4] References

[1] Thingsboard, "ThingsBoard - Open-source IoT Platform," ThingsBoard, https://thingsboard.io/ (accessed Feb. 13, 2026).

[2] FIWARE, https://www.fiware.org/ (accessed Feb. 13, 2026).

[3] J. Bryant, "Why your weather app might be wrong: The truth about weather forecasting," John Bryant, https://weatherandclimateexpert.com/why-your-weather-app-might-be-wrong-the-truth-about-weather-forecasting/ (accessed Feb. 13, 2026).

[4] “App startup time,” Android Developers, https://developer.android.com/topic/performance/vitals/launch-time (accessed Feb. 13, 2026).

[5] City of Toronto, “Toronto at a glance,” City of Toronto, https://www.toronto.ca/city-government/data-research-maps/toronto-at-a-glance/ (accessed Feb. 13, 2026).

[1.5] Overview

Section 2 discusses a higher-level overview of the product, talking about the product perspectives and functions, important information on the users, project constraints, assumptions, and dependencies, and a tentative list of the future requirements. Section 3 contains the use case diagram outlining key use cases for the application. Section 4 lists the main functional requirements aggregated to outline major business events and viewpoints of the platform. Section 5 contains non-functional requirements. These requirements highlight the standards for look and feel, usability and humanity, performance, operational and environmental, maintainability and support, security, cultural, political, and legal. Finally, Section 6 describes the city ranking system, our innovative feature. Appendix A contains the division of labour.

---

[2] Overall Product Description

[2.1] Product Perspective

SCEMAS is a cloud-native IoT software platform that provides a web interface for city operators and a read-only view for the public. Related products include: ThingsBoard, an open source IoT platform for data collection, processing, visualization, and device management (also uses **MQTT,** CoAP, and HTTP, and supports cloud & on-premises deployments [1]. FIWARE is a framework based on supplying a cornerstone function required in any smart solution by managing context information (updates and access) [2]. These products are more for general use; our product will focus specifically on the city environment. The dashboard will include a geographical map of sensor locations, charts of environment metrics, gauges displaying current values, system health, active alerts, and access to historical data trends. The public view is read-only and contains non-sensitive environment data for anyone to see. Similar to ThingsBoard and FIWARE, the software will use industry-standard protocols such as **MQTT** to ensure

security and privacy. Access to the platform will be governed by a role-based access control (RBAC) so that certain data is not tampered with by those who should not be updating it. To ensure this, all users must register an account, which will then be used to check whether a user can act (maybe include how they can register or smth).

The product will interact with the larger system of a real-world mapping software, Google Maps, to display the location of the alert sent by a sensor in the city. Additionally, each alert will contain details about the data the sensor is receiving.

Furthermore, the web dashboard for city operators will include suggested actions when receiving an alert to help them deal with the alert more efficiently.

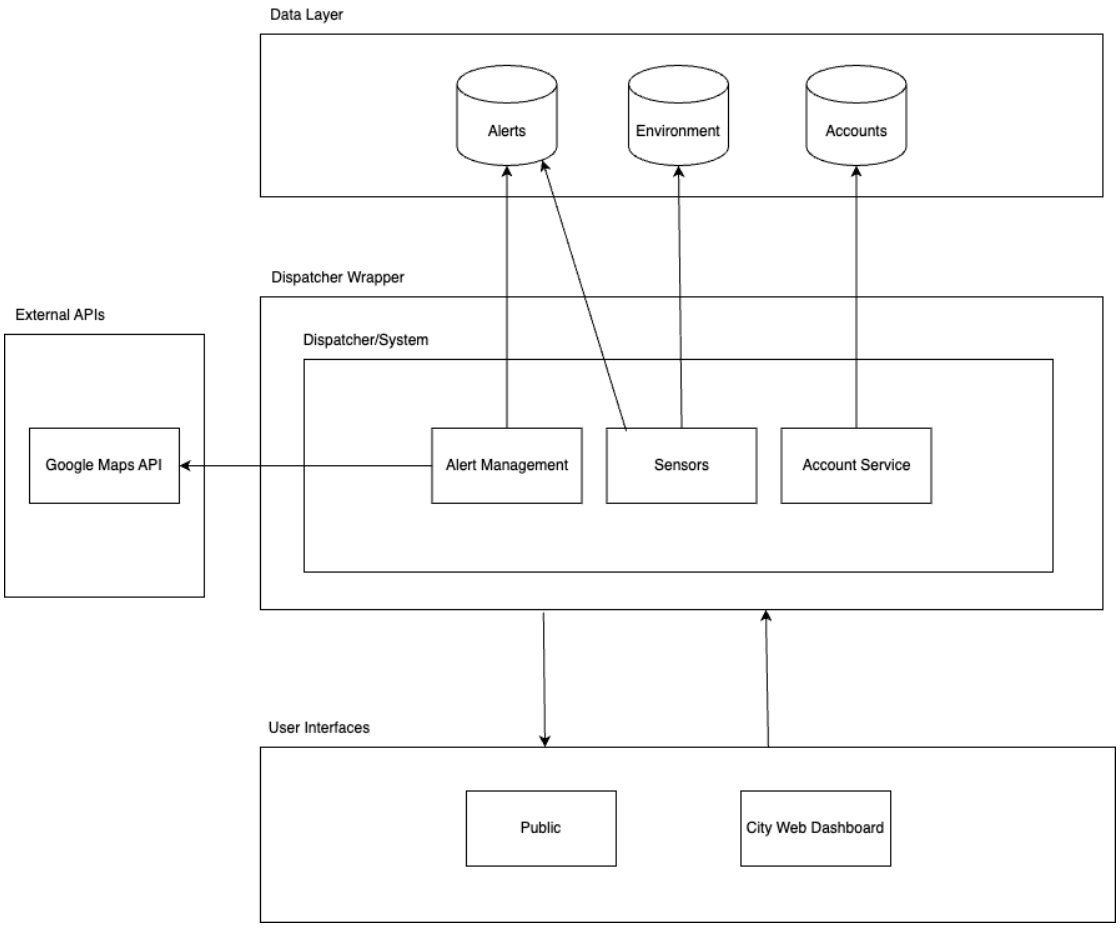

Figure 1: System Diagram

[2.2] Product Functions

Modules	Functions
Alert Manager	● Accept sensor information ○ Receive and unpack information from the sensor. ● Process data ○ Perform operations to clean data into a usable form. ● Aggregate data ○ Perform operations to summarize and aggregate data when the user requests specific filters. ● Detect alerts ○ Monitor incoming data, raise alerts when the criteria for various alerts have been met. ● Send alert ○ Notify users in the case of emergencies or when alarming information is received from the sensors. ● Determine alert destination ○ Based on the alert's severity, determine which users should be notified (by distance, administrative role, etc.). ● Log executions ○ Log information about executed alerts and the application's status to debug crashes and more. ● Rank Cities ○ Apply our innovative feature by aggregating data and ranking them.
Sensors	● Collect Data ○ Sensors must get specified information from the environment. ● Send Data ○ Send the data from the sensor to the alert manager with important metadata.
Account Service	● Create accounts ○ Allows the user to create an account. ● Update account information ○ Allows the user to update their account. ● Login ○ Allows users to log in/log out of their accounts. ● Determine permissions ○ Assign the user the appropriate permissions. ● Encrypt User Information

	○ Hide sensitive information from the user. ● Log executions ○ Log information about the user activities and actions to debug crashes and support audits.

[2.3] User Characteristics

1. Municipal Employees:
Municipal employees who will be tasked with monitoring the weather using **SCEMAS**.

Education Level: Bachelors
Experience: Experience with city planning and other city-related regulations.
Technical Expertise: Moderate. Municipal employees should be trained on how to use **SCEMAS** and solve very minor issues by themselves.

2. Managers and Decision Makers:
Managers and decision makers who will look at the summarized data presented by the municipal employees and make decisions accordingly.

Education Level: Bachelors
Experience: Experience with city planning and other city-related regulations. Furthermore, experience managing employees and experience making decisions regarding city regulations.
Technical Expertise: Moderate. Managers should know more than employees, as they will have more access to functionalities.

3. General Public:
General Public who will use **SCEMAS** for their own personal use

Education Level: Varies, no education required
Experience: No experience required. However, they should know basic weather terminology.
Technical Expertise: Low, General public should have used devices before and are familiar with weather apps, and can navigate through **SCEMAS**

3. IT Administration:
Responsible for maintaining and deploying. Their job is to ensure **SCEMAS** is running and fix any issues that arise.

Education Level: Bachelor's in Computer Science or Engineering
Experience: Experience with software infrastructure, database, IoT systems, and general software development practice
Technical Expertise: High, should be capable of debugging issues, deploying new updates, and ensuring that **SCEMAS** is running.

[2.4] Constraints

1. **Budget:** The budget of the project will have a great impact on what technologies the developer can make use of and ultimately the features that can be inserted into the app. The indicated budget for the project is 0 dollars.

2. **Project Time:** The time allotted for the project can affect how large it will be. As well, it affects the development timeline and the resources available.

[2.5] Assumptions and Dependencies

- The city workers will regularly access the dashboard to view sensor status.
- Workers are trained and able to operate and understand the dashboard.
- The sensors will provide reliable JSON data in a predictable format.
- The position of the sensors is known.
- The sensors deliver data in a predictable and manageable schedule.
- City workers, developers, and other users have different goals using the application.
- The solution will be piloted in only one city/country

Other assumptions that, if they fail to hold, could require a change to the requirements

- Users will have internet access when using the application.
- Assume the sensors will have a reasonable uptime.
- Assume the sensors will be able to collect and transmit information in inclement conditions.
- Assume the APIs are fully functional during the operation of the product.
- Assume the APIs are safe and secure.
- Assume the workers will have access to a device running either an Android or iOS operating system.

[2.6] Apportioning of Requirements

- **Language**: For now, **SCEMAS** will only be available in English
- **Trend Prediction**: **SCEMAS** will have a feature where it will use previous data and predict future occurrences
- **Public User Accounts:** In later releases, the general public will be able to create their own account and set personal preferences
- **Data Exportation:** In later versions, the Municipality will be able to export stored data regarding the weather for analysis

---

[3] Use Case Diagram

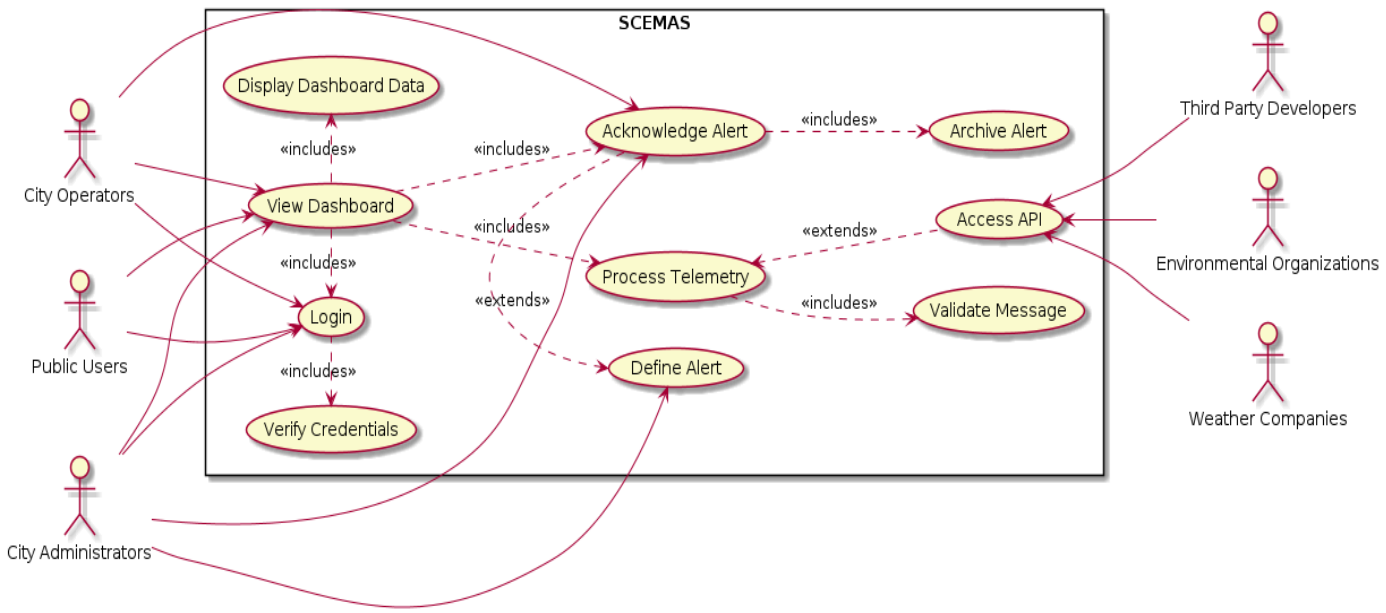

Figure 2: Use case diagram

---

[4] Highlights of Functional Requirements

Main Business Events:

- BE1 Process sensor **telemetry**
- BE2 Login

- BE3 View dashboard
- BE4 Acknowledge alert
- BE5 Access public API
- BE6 Define rule-based alert

Viewpoints:

- VP1 Public Users
- VP2 City Operators
- VP3 City Administrators
- VP4 Third Party Developers
- VP5 Environmental Organizations
- VP6 Weather Companies

BE1. Process sensor **telemetry** #1

Pre-condition: Sensor devices are deployed around the city.

VP1. Public Users #1

Main success scenario:

1. Sensor broadcasts **telemetry** via the **MQTT** protocol.
2. The system receives and validates the message.
3. The system stores the **telemetry** in a time-series database.
4. The system performs real-time aggregation on the data.
5. Public users can view the aggregated data on the public home page.

Secondary success scenario:

2i. System fails to validate the message and rejects the **telemetry** data.

VP2. City Operators #2

5. Operators can view aggregated **telemetry** and real-time visualizations of data on the dashboard.

VP3. City Administrators #3

5. Administrators can view aggregated **telemetry** and real-time visualizations of data on the dashboard.

VP4. Third Party Developers #4

5. Developers can access up-to-date aggregated data through the public **REST API.**

VP5. Environmental Organizations #5

NA

VP6. Weather Companies #6

NA

Global Scenario:

Pre-condition: Sensor devices are deployed around the city.

Main success scenario:

1. Sensor broadcasts **telemetry** via the **MQTT** protocol.
2. The system receives and validates the message.
3. The system stores the **telemetry** in a time-series database.
4. The system performs real-time aggregation on the data.
5. Users can access appropriate data available through the public interface, the worker dashboard, or the **REST API**.

Secondary success scenario:

2i. The system fails to validate the message and rejects the **telemetry** data.

BE2. Login #2

Pre-condition: User possesses a system email and password

VP1. Public Users #1

Main success scenario:

1. System requests the username and password.
2. User enters information and presses “Login”.
3. System verifies email and password.
4. The system proceeds to the public home page.

Secondary success scenario:

3i. Email and password are invalid

3i.1 The system displays an error message.

3i.2 System prompts user to re-enter data.

VP2. City Operators #2

4. The system proceeds to the operation home page.

VP3. City Administrators #3

5. The system proceeds to the administration home page.

VP4. Third Party Developers #4

NA

VP5. Environmental Organizations #5

NA

VP6. Weather Companies #6

NA

Global Scenario:

Pre-condition: User possesses a system email and password

Main success scenario:

1. System requests the user's email and password.
2. User enters information and presses “Login”.
3. System verifies email and password.
4. The system proceeds to the public/operation/administration home page.

Secondary success scenario:

3i. Email and password are invalid

3i.1 The system displays an error message.

3i.2 System prompts user to re-enter data.

BE3. View Dashboard

Pre-condition: User has valid credentials and is logged in

VP1. Public Users #1

N/A

VP2. City Operators #2

Main success scenario:

1. System request the user’s email and password to log in
2. System authenticates user and gives them access accordingly
3. System displays an operational dashboard
4. User views current data on the geographical map
5. User views active alerts
6. User views historical data and trends
7. User closes the dashboard

Secondary Success Scenario:

3i. System fails to load dashboard data

3i.1 System displays error messages and prompts user to refresh

VP3. City Administrators #3

1. User accesses logs and rule-based configurations

VP4. Third Party Developers #4

N/A

VP5. Environmental Organizations #5

N/A

VP6. Weather Companies #6

N/A

Global Scenario:

Pre-condition: User has valid credentials and is logged in

Main success scenario:

1. System request the user’s email and password to log in

2. System authenticates user and gives them access accordingly
3. System displays an operational dashboard
4. User views current data on a geographical map
5. User views active alerts
6. User views historical data and trends
7. User closes dashboard

Secondary Success Scenario:

3i. System fails to load dashboard data

3i.1 System displays error messages and prompts user to refresh

BE4. Acknowledge Alert #4

Precondition: The user must have an account.

VP1. Public Users #1

N/A

VP2. City Operators #2

Main Success Scenario

1. User opens the **SCEMAS** website on their work computer.
2. System requires the user to log in and displays the log-in fields.
3. User enters login details.
4. System authenticates the user and gives them the access their account has.
5. System displays an alert.
6. User clicks on the alert.
7. System displays alert information.
8. User marks the alert as finished.
9. System closes the alert.
10. System archives the alert.
11. System sends a notification to every user about the alert being finished.

Secondary Scenario

4i. System fails to authenticate the user

4i.1. System fails to authenticate the user.

4i.2. System re-prompts user to enter login details.

9i. System fails to close alert.

9i.1. System fails to close alert.

9i.2. Acknowledge Alert failed.

10i. System fails to archive alert.

10i.1. System fails to archive alert.

10i.2. Acknowledge Alert failed.

11i. System fails to send a notification.

11i.1. System fails to send a notification.

11i.2. Acknowledge Alert failed.

VP3. City Administrators #3

8. User edits the alert details.
9. System updates the alert details globally.

9i. System fails to update the alert

VP4. Third Party Developers #4

N/A

VP5. Environmental Organizations #5

N/A

VP6. Weather Companies #6

N/A

Global Scenario:

Precondition: The user must have an account.

Main Success Scenario

1. User opens the **SCEMAS** website on their work computer.
2. System requires the user to log in and displays the log-in fields.
3. User enters login details.
4. System authenticates the user and gives them the access their account has.
5. System displays an alert.
6. User clicks on the alert.
7. System displays alert information.
8. User can perform the appropriate actions they have access to.
9. System performs necessary updates to the alert.
10. System archives the alert.
11. System sends a notification about changes to the alert.

Secondary Scenario

4i. System fails to authenticate the user

4i.1. System fails to authenticate the user.

4i.2. Return to BE4-2. (retry acknowledge)

8i. User edits alert details.

9i. System fails to perform necessary updates.

9i.1. System fails to update the alert.

9i.2. Acknowledge Alert failed.

10i. System fails to archive alert.

10i.1. System fails to archive alert.

10i.2. Acknowledge Alert failed.

11i. System fails to send a notification.

11i.1 System fails to send a notification.

11i.2 Acknowledge Alert failed.

BE5. Access Public API #5

Precondition: The SCEMA backend services are operational, and the public **REST API** endpoint is exposed securely.

VP1. Public Users #1

N/A

VP2. City Operators #2

N/A

VP3. City Administrators #3

N/A

VP4. Third Party Developers #4

Main Success Scenario:

1. Developer accesses the **SCEMAS** Public API documentation.
2. Developer writes a script to request specific sensor averages for a chosen zone.
3. System validates the request schema and rate-limit status.
4. System returns a 200 OK status and the requested time-series data

Secondary Scenario:

3i. Invalid parameters provided

3i.1 Developer requests a zone ID that does not exist

3i.2 System returns a 404 Not Found error.

VP5. Environmental Organizations #5

Main Success Scenario:

1. The organization sets up an automated integration to collect daily pollution summaries.
2. The organization's system calls the Public API on a scheduled basis.
3. The organization uses the data to generate independent sustainability reports.

VP6. Weather Companies #6

Main Success Scenario:

1. Weather company integrates **SCEMAS** hyper-local temperature data into their forecasting model.

2. Company API client requests high-frequency updates from multiple zones.
3. The system serves the data, ensuring no sensitive information or precise sensor coordinates are revealed.

Global Scenario:

Main Success Scenario:

1. Client sends an HTTPS GET request to the Public API endpoint.
2. System verifies that the request does not violate security or rate-limiting policies.
3. System queries the time-series database for the requested aggregated data.
4. System masks any sensitive information (e.g., precise sensor coordinates).
5. System returns a JSON response containing the requested environmental metrics.

Secondary Success Scenario:

2i. Invalid parameters provided

2i.1 Client requests a zone ID that does not exist

2i.2 System returns a 404 Not Found error.

BE6. Define rule-based alert #6

Pre-condition: User is logged in and has been verified as an administrator.

VP1. Public Users #1

N/A

VP2. City Operators #2

N/A

VP3. City Administrators #3

Main success scenario:

1. Administrator selects “create new alert”.
2. System asks for a rule definition.
3. Administrator enters relevant information.
4. System creates a new rule-based alert.
5. System will now monitor for violations.

Secondary success scenario:

3i. Incorrect information.

3i.1 Incorrect information.

3i.2 Display an error message, and request new information.

VP4. Third Party Developers #4

N/A

VP5. Environmental Organizations #5

N/A

VP6. Weather Companies #6

N/A

Global Scenario:

Pre-condition: User is logged in and has been verified as an administrator.

Main success scenario:

6. Administrator selects “create new alert”.
7. System asks for a rule definition.
8. Administrator enters relevant information.
9. System creates a new rule-based alert.
10. System will now monitor for violations.

Secondary success scenario:

3i. Incorrect information.

3i.1 Incorrect information.

3i.2 Display an error message, and request new information.

---

[5] Non-Functional Requirements

[5.1] Look and Feel Requirements

5.1.1 Appearance Requirements

LF-A1. The system must avoid using bright and saturated colours as backgrounds on any screen.

Rationale: The system will use those colours for urgent and important parts of the system, such as alerts.

LF-A2. The system shall use a minimalist design.

Rationale: A minimalist design features an interface that is simple and clear. Every element has a purpose, whether that be a button with a corresponding function or just a text box explaining something. The design must also make good use of negative space so the elements are not cluttered and are easier to identify. This will make it easier to navigate the system. Also, universally understood symbols such as an “X” to close something will be used instead of using the word “Close” as having too many words can cause clutter.

LF-A3. The system must have different, distinct colours for each element.

Rationale: Having a uniform colour for each element will make it easier to identify the type of element. For example, if all buttons are green, the user can clearly identify where all the buttons on the page are. This will also improve system navigation.

LF-A4. The system must use 1 consistent font choice throughout the app.
Rationale: Having a consistent font will make the system easily readable and easier to navigate.

5.1.2 Style Requirements

LF-S1. The system must have a navigation hierarchy so that all content is available within 3 clicks of the starting page.
Rationale: Having more than 3 clicks to get to one page is tedious and may be too time-consuming when dealing with urgent matters, such as handling alerts. This improves system navigation.

LF-S2. The system must scale to screen size
Rationale: The system should scale to ensure that all devices get the same experience and the designs are consistent among devices. This will allow users to easily adjust when switching between devices, also making the app more flexible.

LF-S3. The system must indicate an alert through an indicator that appears on every page of the app.
Rationale: The indicator will tell the user that there is an alert. If the user is not on the page to handle the alert, they should still be notified of one and have an option to jump to that page for quick access.

[5.2] Usability and Humanity Requirements

5.2.1 Ease of Use Requirements

UH-EOU1. The system must allow users to provide feedback through the application.
Rationale: Users must have a way to give feedback so the app can be improved. An in-app reporting tool would encourage users to provide more feedback.

UH-EOU2. A trained city operator must be able to acknowledge an alert within 30 seconds.

Rationale: Rapid interaction with the dashboard is critical for time-sensitive events.

5.2.2 Personalization and Internationalization Requirements

UH-PI1. The system must allow users to select a preferred language.

Rationale: Users should be able to fully understand all text-based interfaces. Allowing multiple languages will allow users to be more comfortable on the app.

UH-PI2. The system must allow users to select between metric and imperial units

Rationale: Some users will be more familiar with different units and must be able to select the units they are most comfortable with.

5.2.3 Learning Requirements

UH-L1. Public and third-party developers must be able to learn to access the API within a two-hour time frame.

Rationale: The API should be straightforward to learn and use for public and third-party developers.

5.2.4 Understandability and Politeness Requirements

NA

5.2.5 Accessibility Requirements

UH-A1. The system must be colour blind compatible.

Rationale: Users who are colour blind should be able to differentiate all parts of the interface.

[5.3] Performance Requirement

5.3.1 Speed and Latency Requirements

PR-SL1. The sensor response time must be less than 5 minutes.

Rationale: Current models only update every 6 to 12 hours [3]. The goal of this system is to inform users about their local environment in an accurate and timely manner. That is why the system needs the most recent sensor data it can get.

PR-SL2. The system startup time should be less than 1.5 seconds.

Rationale: Android vitals states that 1.5 seconds or longer is considered an excessive startup time [4]. Our system must comply with the industry standard. Also, the chances of losing users go up if the startup time is long. This may encourage users to switch to another app. This works both ways; having a fast startup time may attract users from other competing apps to this one.

5.3.2 Safety-Critical Requirements

N/A

5.3.3 Precision or Accuracy Requirements

PR-PA1. The system must have an overall accuracy rate greater than 95%.

> **Rationale:** The system must be accurate, or else people will not use it. The system's main purpose is to alert to weather conditions; people are interested in the real values, not wrong values.

5.3.4 Reliability and Availability Requirements

PR-RA1. The system must have an uptime of 99%.

> **Rationale:** The system is an alerting system, meaning it should always report critical events as soon as possible. Because the environment is random, in order to respond as soon as possible, the system must be up most of the time. Failing to do so will reduce the number of users.

PR-RA2. The system must create backups of all the alerts and who handled them.

> **Rationale:** The system has a human operator (city worker) who can either increase or decrease the reliability of the system. Keeping backups of who did what creates accountability for one's actions. This will reduce the chances of operators sending fake alerts or closing alerts when they shouldn’t be closed.

5.3.5 Robustness or Fault-Tolerance Requirements

PR-RFT1. The system must be able to handle exceptional inputs made by the user.

> **Rationale:** If the input is exceptional, then the system should not crash, but instead let them know what went wrong and suggest actions.

5.3.6 Capacity Requirements

PR-C1. The system should be able to handle at least 3 million concurrent users.

> **Rationale:** Toronto has the largest city population in Canada, with more than 3 million people living there [5]. Assuming everyone uses the system, the system should still work with that many concurrent users. This will ensure that the system will still be functional in every city in Canada.

5.3.7 Scalability or Extensibility Requirements

N/A

5.3.8 Longevity Requirements

N/A

[5.4] Operational and Environmental Requirements

5.4.1 Expected Physical Environment

OE-EPE1. The system will operate with sensor devices deployed throughout the city and must tolerate typical city weather conditions.

> **Rationale:** Sensors will be installed outdoors across the city and must be resilient to any weather they are exposed to.

5.4.2 Requirements for Interfacing with Adjacent Systems

OE-IA1. The system must be able to receive information from the Google Maps API.

> **Rationale:** The system will use map information to display sensor **telemetry** in real time visually.

5.4.3 Productization Requirements

OE-P1. The system will be packaged as a web-based application and require no downloads to function.

> **Rationale:** A web-based system will be the most available for the largest number of users.

5.4.4 Release Requirements

OE-R1. Bug fixes and security issues shall be patched with new releases as needed.

> **Rationale:** Issues with the system must be addressed and patched.

OE-R2. API updates shall maintain backward compatibility.

> **Rationale:** Third-party developers may rely on past versions of the API, and behaviour should be consistent.

[5.5] Maintainability and Support Requirements

5.5.1 Maintenance Requirements

MS-M1. The system shall be designed to decouple sensors and processing.

Rationale: Modularity ensures that the individual components (alerting engine, data pipeline, new sensors, remove sensors, etc.) can be updated or refactored without taking the entire system offline.

5.5.2 Supportability Requirements

MS-S1. The Public API must be provided alongside extensive documentation that allows the average third-party developer to successfully request and parse data within two hours.

Rationale: The API must be well-documented to mitigate misuse and provide a reasonable development experience with external integrations.

5.5.3 Adaptability Requirements

MS-A1. The system must support remote updates to the sensor configurations via a control channel.

Rationale: Systems should be able to be altered and adapted with minimal friction when real-time data is being exchanged to ensure all systems are updated within the smallest time frame possible.

[5.6] Security Requirements

5.6.1 Access Requirements

SR-AC1. System must implement role-based controls and features

Rationale: Some features should only be available to system administrators or city operators

5.6.2 Integrity Requirements

SR-INT1. All data transfer should be encrypted

Rationale: Having encrypted data is necessary as it protects the user's data from being stolen.

5.6.3 Privacy Requirements

SR-P1. The system should not store any unnecessary personal information of users.

Rationale: Ensure no unnecessary data is leaked

SR-P2: Data of general zones, such as neighbourhood or city, is available rather then exact locations of the users

Rationale: Prevents tracking of users but still provides useful data of the surrounding environment

5.6.4 Audit Requirements

SR-AU1. The system must maintain all important logs and changes made to the rule configurations, alerts, or updates.

Rationale: Helps with transparency between the admin and IT team and the users of **SCEMAS**

5.6.5 Immunity Requirements

SR-IM1. The system must not accept unexpected input from users or sensors

Rationale: This is to prevent attacks

[5.7] Cultural and Political Requirements

5.7.1 Cultural Requirements

CP-C1. The system shall provide data and information using standardized, culturally recognized indices (e.g., Air Quality Index - AQI).

Rationale: The information needs to be provided in a way that is instantly recognizable to maximize usage from the general public. This information should be understood regardless of technical background.

5.7.2 Political Requirements

CP-P1. The system must expose non-sensitive environmental data to the public and third-party developers via a read-only **REST API** to uphold open data transparency.

Rationale: Smart city initiatives are funded by the taxpayer; transparency is not negotiable. Providing easily accessible services to those who request (and fund) the information fulfills the project's vision.

[5.8] Legal Requirements

5.8.1 Compliance Requirements

LR-COMP1. The system shall not collect, process, or store any personally identifiable information.

Rationale: This is a mandatory constraint to ensure compliance with privacy standards and reduce legal liability in case of cyber attacks or data leaks.

LR-COMP2. All environmental **telemetry** must be collected with broad city zones rather than precise GPS coordinates.

Rationale: Associating data with zones instead of precise locations is a core "privacy-by-design" principle required for this system.

5.8.2 Standards Requirements

LR-STD1. All data in transit must be encrypted using industry-standard protocols such as TLS or MQTTS.

Rationale: Strong authentication and encryption are inherently included when designing a private asn secure system to meet professional security standards associated with dealing with sensitive location data.

LR-STD2. The public **REST API** must implement rate-limiting controls.

Rationale: These controls are required to mitigate denial-of-service (DoS) attacks and prevent unauthorized data scraping.

---

[6] Innovative Feature

The chosen innovative feature to implement is called the city ranking system. It is a menu that ranks environmental variables across all cities in Canada, with the number 1 being the best in terms of quality. Users will also be able to filter which variable(s) to compare and which cities to compare. Finally, they will be able to search for a specific city to view the rank of all the variables of that specific city.

This would be a convenient feature for those who are looking to move to a new city or those just wondering about the state of their current location compared to others. For example, users looking to move can check variables such as the air quality to see how it compares to other cities before choosing a city to move to. A ranking system also targets those who do not live in a city for the same reasons stated above, which may increase the number of users using the product.

A Division of Labour

Heathfield, Aidan:

- Helped with 1 Introduction
- Helped with 1.1 Purpose
- Helped with 1.2 Scope
- 1.4 References
- 1.5 Overview
    - I updated the line about the innovative feature
- 2.1 Product Perspective
- 2.4 Constraints
- 4 Highlights of Functional Requirements
    - I did BE4
- 5.1 Look and Feel Requirements

- 5.3 Performance Requirements
- 6 Innovative Feature

McEachern, Connor:

- Helped with 1 Introduction
- Helped with 1.1 Purpose
- 1.3 Definitions, Acronyms, and Abbreviations
- Helped with 1.5 Overview
- 2.2 Product Functions
- 3 Use Case Diagram
- 4 Highlights of Functional Requirements
    - BE 1
    - BE 2
- 5.2 Usability and Humanity Requirements
- 5.4 Operational and Environmental Requirements

Saumy, Patel:

- Helped with 1 Introduction
- Helped with 1.1 Purpose
- Helped with 1.2
- 2.3
- 2.6
- 4 Highlights of Functional Requirements
    - BE3
- 5.6

Zafar, Ahmed:

- Helped with 1 Introduction
- Helped with 1.1 Purpose
- Helped with 1.2 Scope
- Helped with 1.3 Definitions, Acronyms, and Abbreviations
- 1.5 Overview
- 2.2 Product Functions
- 2.5 Assumptions and Dependencies
- 4 Highlights of Functional Requirements
    - BE4
- 5.5 Maintainability
- 5.7 Cultural and Political Requirements
- 5.8 Legal Requirements

Deliverable #2: Preliminary Architecture Analysis

SE 3A04: Software Design II - Large System Design

Tutorial: T03
Group: G2
Group Members:
McEachern, Connor Joshua
Zafar, Ahmed
Heathfield, Aidan Amato
Patel, Saumy Piyushbhai

[1] Introduction:

[1.1] Purpose:

This document provides a high-level overview of the **SCEMAS** Smart City Environmental Monitoring & Alert System, including high-level design considerations of the system, as well as design considerations for various subsystems.
This document is intended for internal **SCEMAS** stakeholders, including but not limited to project managers, developers, domain experts, and **SCEMAS** team members/investors. **SCEMAS** Deliverable 1 should be read before, and technical knowledge may be beneficial in better understanding the contents of the document.

[1.2] System Description:

The system follows a Model-View-Controller (MVC) style architecture, with various subsystems following a repository style and pipe-filter design architecture. A MVC style architecture supports multiple views synchronized with the same data model, and is flexible to support frequent data changes and user interface updates.

A repository-style design architecture was chosen for the subsystems, as it supports concurrent access to data repositories (i.e., alerts, accounts), and is specialized for high-complexity data processing required for **SCEMAS,** since there are many different forms of data being processed.

The pipe-filter style design architecture was also chosen for the subsystems, as it supports filtering data from the sensors, aggregating only what the system uses.

[1.3] Overview

Section 2 presents an Analysis Class Diagram for **SCEMAS**. In Section 3, the document dives deeper into the architectural design of the system, as well as discusses considered alternatives and the reasoning behind design decisions. Finally, in Section 4, Class Responsibility Collaboration (CRC) Cards are presented to dive deeper into classes, responsibilities, and interactions between classes.

[2] Analysis Class Diagram

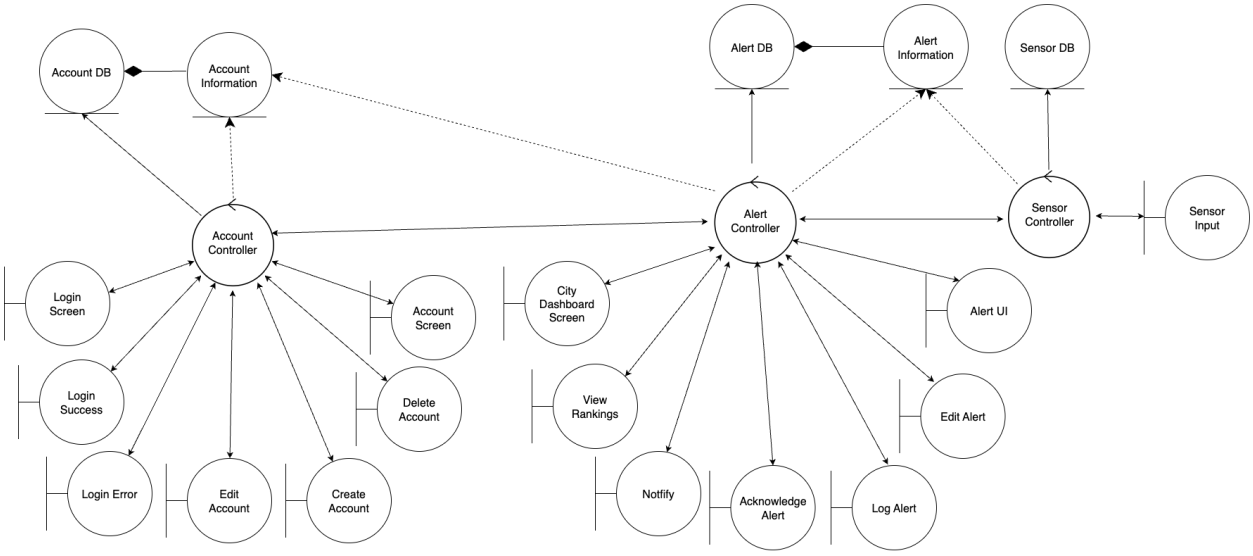

Figure 1. Analysis Class Diagram

[3] Architectural Design

[3.1] System Architecture:

SCEMAS uses the Model View Controller (MVC) architecture style. The view represents the dashboard that City Operators and Administrators will use to create and respond to alerts. The model and controller represent the telemetry data and alert rules that will be collected and stored, as well as the account service.

Within the controller, several different subsystems are defined as follows:

Subsystem	Purpose	Architecture Style
Alert Manager	Create, monitor, and detect alerts and alert rules	Repository
Account Service	Create and log in to accounts	Repository
Sensors (Information handler)	Accept, validate, aggregate, and store incoming telemetry data	Pipe and Filter

The complete architecture of the system incorporates the Model View Controller (MVC), the Repository, and the Pipe and Filter Architecture Styles. MVC is used because it

supports interaction between the administrators who create alert rules and the operators who respond, all within a synchronized data model for storing telemetry data. This is useful because sensors can report telemetry, and the system can respond in real time. MVC supports frequent data changes, which is useful because the dashboard must continuously update in response to incoming sensor data. Additionally, dashboard views can be created and modified without requiring changes to any internal systems.

The Repository Architecture Style is used for the Account Service and Alert Manager. Using a data-centred architecture is beneficial because these subsystems must store and process large volumes of information coming from different sources, including user input and incoming sensor data. The repository style supports the implementation of relational database management systems, which both subsystems will use to manage, query, and maintain data. In this architecture, all important data is accessible to all components that require it while maintaining data consistency across the system. The Repository Architecture makes it simple to store and manage all of the different data that will be collected by the system.

The final architecture style used in the system is the Pipe and Filter architecture style, which is implemented within the Information Handler. The Information Handler is responsible for accepting incoming telemetry from the sensors, validating its correctness, and aggregating similar data. This architecture is well-suited because each stage of telemetry processing can be separated into independent filters that perform specific operations on the data before passing it to the next stage. This design enhances modularity, enabling individual steps to be updated or modified without impacting the rest of the system.

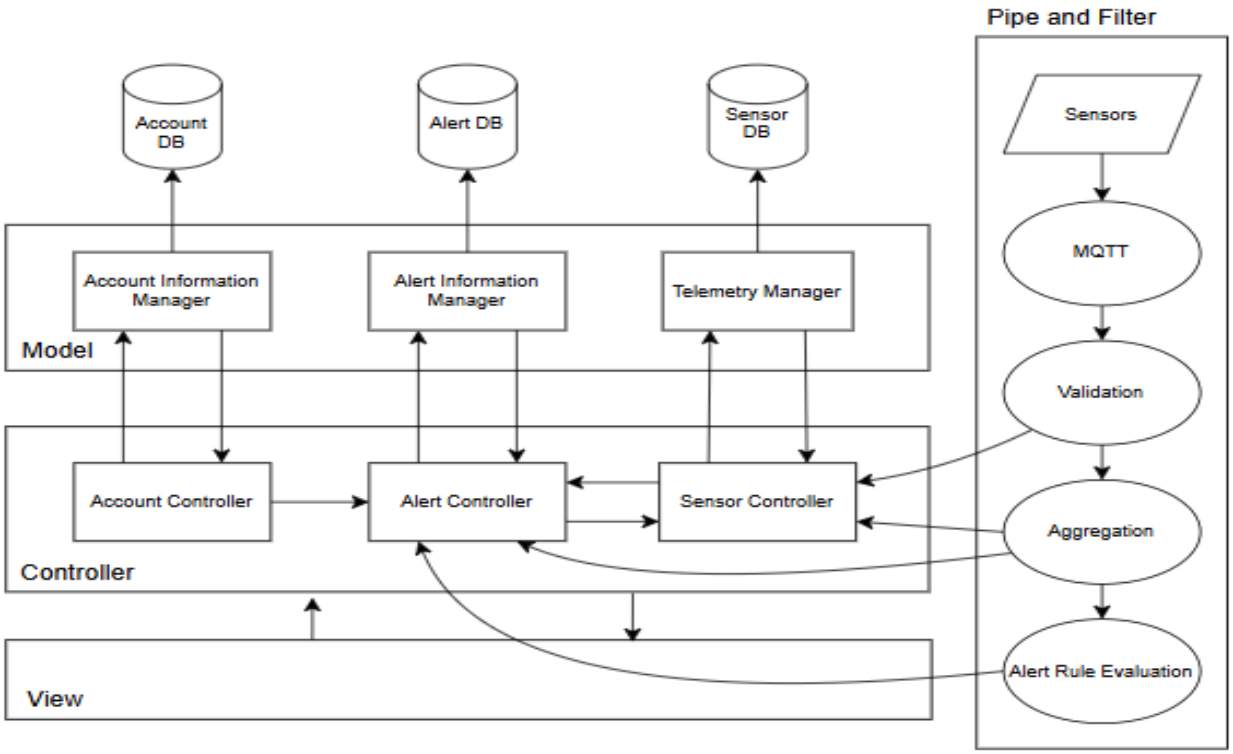

Figure 2. System Architecture

We considered Batch Sequential, but decided that it would be easiest to handle incoming data separately, rather than batching it and spending it all together. Pipe and Filter allows for more throughput from our sensors and does not force sensor data to be coupled together.

The Process-Control Architecture Style is typically used for the control of embedded systems and is not relevant for SCEMAS.

Presentation-Abstraction-Control (PAC) Architecture was not used because the dashboard in the system does not require a layered view. The system is required to have a synchronized view for all users and does not require multiple agents displaying different views.

The Blackboard Architecture Style was not used because the system requires alerts to be deterministic and clearly defined. In a blackboard system, multiple agents contribute knowledge dynamically, which could make alert outcomes unpredictable. Since our alerts must trigger based on specific rules without relying on collaborative input, the blackboard approach is not suitable.

[3.2] Subsystem Description:

Our main system is divided into 3 subsystems: Alert Manager, Account Service, and Sensors (Information handler)

The Sensor subsystem of SCEMAS encompasses the Pipe and Filter to process incoming environmental data. Sensors using IoT are deployed throughout the city, collecting crucial data such as wind speed, precipitation, humidity, and more, as well as writing them to the database via the MQTT protocol. Once received, the data is validated, aggregated, and stored in a time series database, ensuring clean structure and efficiency.

The Alert Management uses a repo architecture style to store weather data and evaluate weather alert rules. It evaluates real-time data initially received by the Sensors and compares them to weather and trend conditions. Using pre-determined thresholds, it determines if a certain value is an alert. Upon detecting a violation of the conditions, it triggers an alert, displays it on the dashboard, logs the events, and notifies external subsystems.

Account services are the foundation for SCEMAS. It is responsible for maintaining account information such as logins, custom settings for each account, and maintaining general security. Built on a repository architecture style, it acts as a management system, ensuring all custom settings and updates to account information are stored. Furthermore, it keeps audit logs of all account-related changes and logins.

Together, these 3 subsystems create a layered architecture. The sensor subsystem retrieves the data, validates and stores it in a time-series database. The Alert Manager then goes through the data and determines which data is an alert. The alert is then logged and notified to individuals with the necessary settings enabled and external subsystems. Furthermore, the account manager is responsible for validating logins and user settings, ensuring the security and privacy of data.

[4] Class Responsibility Collaboration (CRC) Cards

Class Name: Account Controller (Controller)	
Responsibilities: • Knows Account Information • Knows Login Screen • Knows Login Success • Knows Login Error • Knows Edit Account • Knows Create Account • Knows Delete Account • Knows Account Screen • Determines role-based permitted user actions for account management.	**Collaborators:** • Account Information • Login Screen • Login Success • Login Error • Edit Account • Create Account • Delete Account • Account Screen • Alert Controller

Class Name: Account Information (Entity)	
Responsibilities: • Knows Account DB • Knows Account Controller • Knows Alert Controller	**Collaborators:** • Account DB • Account Controller • Alert Controller

Class Name: AccountDB(Entity)	
Responsibilities: • Knows Account Information • Stores account records	**Collaborators:** • Account Information

Class Name: Login Screen(Boundary)	
Responsibilities: • Knows Account Controller • Handles user login events	**Collaborators:** • Account Controller

Class Name: Login Success(Boundary)	
Responsibilities: • Knows Account Controller	**Collaborators:** • Account Controller

• Displays a successful login interface	

Class Name: Login Error (Boundary)	
Responsibilities: • Knows Account Controller • Displays an unsuccessful login interface	**Collaborators:** • Account Controller

Class Name: Edit Account (Boundary)	
Responsibilities: • Knows Account Controller • Handles account modification events	**Collaborators:** • Account Controller

Class Name: Create Account(Boundary)	
Responsibilities: • Knows Account Controller • Handles new account creation events	**Collaborators:** • Account Controller

Class Name: Delete Account (Boundary)	
Responsibilities: • Knows Account Controller • Handles account deletion events	**Collaborators:** • Account Controller

Class Name: Account Screen(Boundary)	
Responsibilities: • Knows Account Controller • Displays primary account interface	**Collaborators:** • Account Controller

Class Name: Alert Controller (Controller)

Responsibilities:	Collaborators:
• Knows Alert Information • Knows Account Controller • Knows Sensor Controller • Knows City Dashboard Screen • Knows View Rankings • Knows Notify • Knows Acknowledge Alert • Knows Archive Alert • Knows Edit Alert • Knows Alert UI • Handles requests that involve alerts • Determines role-based permitted user actions • Interprets incoming sensor data	• Alert Information • Account Controller • Sensor Controller • City Dashboard Screen • View Rankings • Notify • Acknowledge Alert • Archive Alert • Edit Alert • Alert UI

Class Name: Alert Information (Entity)	
Responsibilities: • Knows Alert DB • Knows Alert Controller	**Collaborators:** • Alert DB • Alert Controller

Class Name: Alert DB (Entity	
Responsibilities: • Knows Alert Information • Stores historical and active alerts	**Collaborators:** • Alert Information

Class Name: City Dashboard Screen (Boundary)	
Responsibilities: • Knows Alert Controller • Displays the city dashboard	**Collaborators:** • Alert Controller

Class Name: View Rankings (Boundary)	
Responsibilities:	**Collaborators:**

• Knows Alert Controller • Handles requests to view rankings	• Alert Controller

Class Name: Notify (Boundary)	
Responsibilities: • Knows Alert Controller • Handles sending and displaying notifications	**Collaborators:** • Alert Controller

Class Name: Acknowledge Alert (Boundary)	
Responsibilities: • Knows Alert Controller • Handles alert acknowledgment events	**Collaborators:** • Alert Controller

Class Name: Archive Alert (Boundary)	
Responsibilities: • Knows Alert Controller • Handles alert archiving events	**Collaborators:** • Alert Controller

Class Name: Edit Alert (Boundary)	
Responsibilities: • Knows Alert Controller • Handles alert modification events	**Collaborators:** • Alert Controller

Class Name: Alert UI (Boundary)	
Responsibilities: • Knows Alert Controller • Displays primary alert interface	**Collaborators:** • Alert Controller

Class Name: Sensor Controller (Controller)	
Responsibilities: ● Knows Sensor DB ● Knows Alert Controller ● Forwards applicable and appropriate information to the alert controller	**Collaborators:** ● Sensor DB ● Alert Controller

Class Name: Sensor DB (Entity)	
Responsibilities: ● Knows Sensor Controller ● Stores sensor data readings	**Collaborators:** ● Sensor Controller

---

[A] Division of Labour

Heathfield, Aidan:

- 1.1
- 1.2
- 1.3
- 2

McEachern, Connor:

-

- 1.1
- 1.3

- 3.1

Saumy, Patel:

- 1.1,
- 1.3
- 3.2

Zafar, Ahmed:

-

- Revision of Deliverable 1.
- 1.1,
- 1.3,
- 4

Deliverable #3

SE 3A04: Software Design II - Large System Design

Tutorial: T03
Group: G2
Group Members:
McEachern, Connor Joshua
Zafar, Ahmed
Heathfield, Aidan Amato
Patel, Saumy Piyushbhai

[1] Introduction:

[1.1] Purpose:

This document provides further information about the **SCEMAS** Smart City Environmental Monitoring & Alert System architecture, including state chart diagrams, sequence diagrams, and a detailed class diagram.

This document is intended for internal SCEMAS stakeholders, including but not limited to project managers, developers, domain experts, and SCEMAS team members/investors. SCEMAS Deliverables 1 and 2 should be read prior to, and technical knowledge may be beneficial in better understanding the contents of the document.

[1.2] System Description:

The system follows a Model-View-Controller (MVC) style architecture, with various subsystems following a repository style and pipe-filter design architecture. A MVC style architecture supports multiple views synchronized with the same data model, and is flexible to support frequent data changes and user interface updates.

A repository-style design architecture was chosen for the subsystems, as it supports concurrent access to data repositories (i.e., alerts, accounts), and is specialized for high-complexity data processing required for **SCEMAS,** since there are many different forms of data being processed.

The pipe-filter style design architecture was also chosen for the subsystems, as it supports filtering data from the sensors, aggregating only what the system uses.

[1.3] Overview

This document is organized by chart/diagram type. Section 2 contains relevant state chart diagrams, Section 3 contains relevant sequence diagrams, and Section 4 provides a detailed UML diagram of the system.

[2] State Charts for Controller Classes

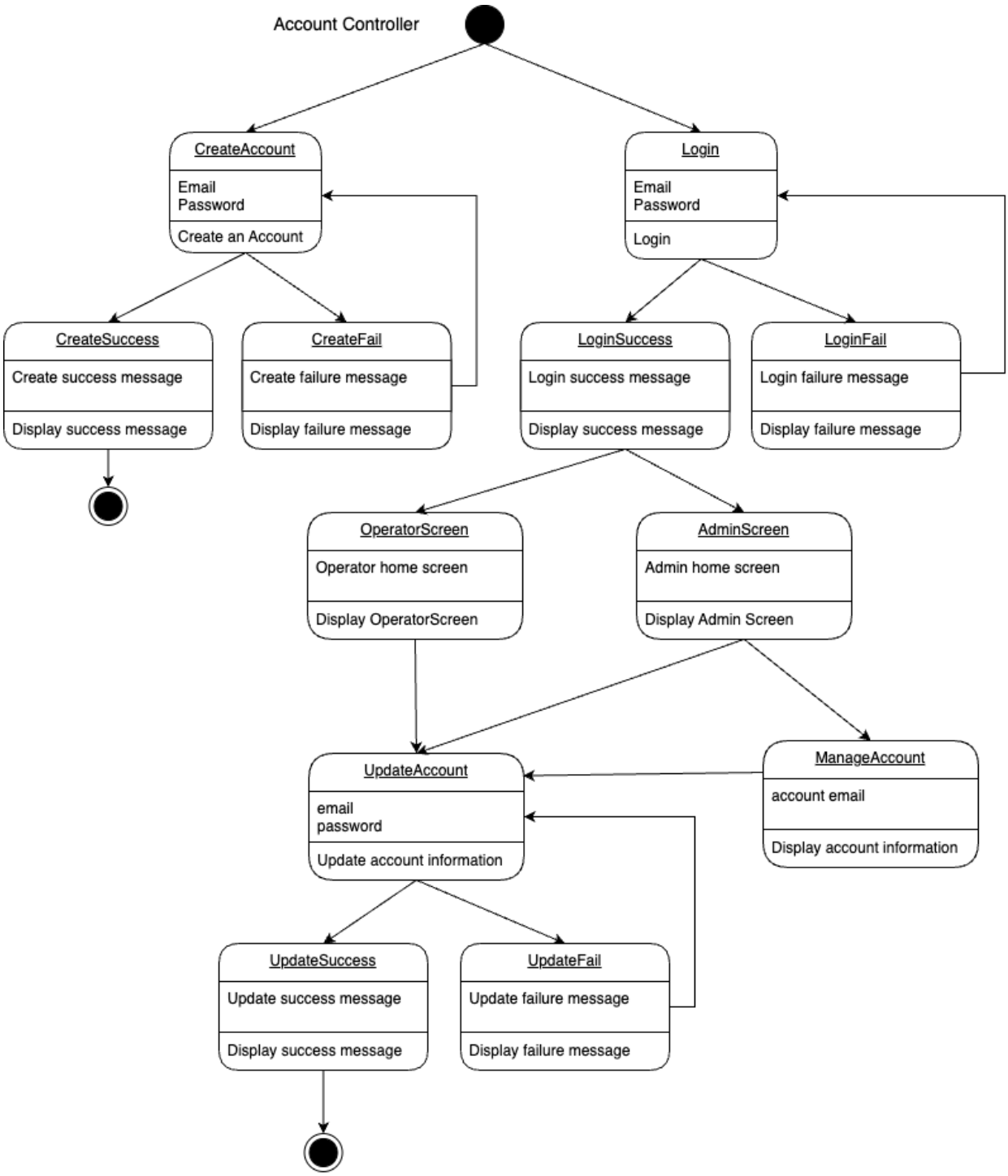

Figure 1 Account Controller State Diagram

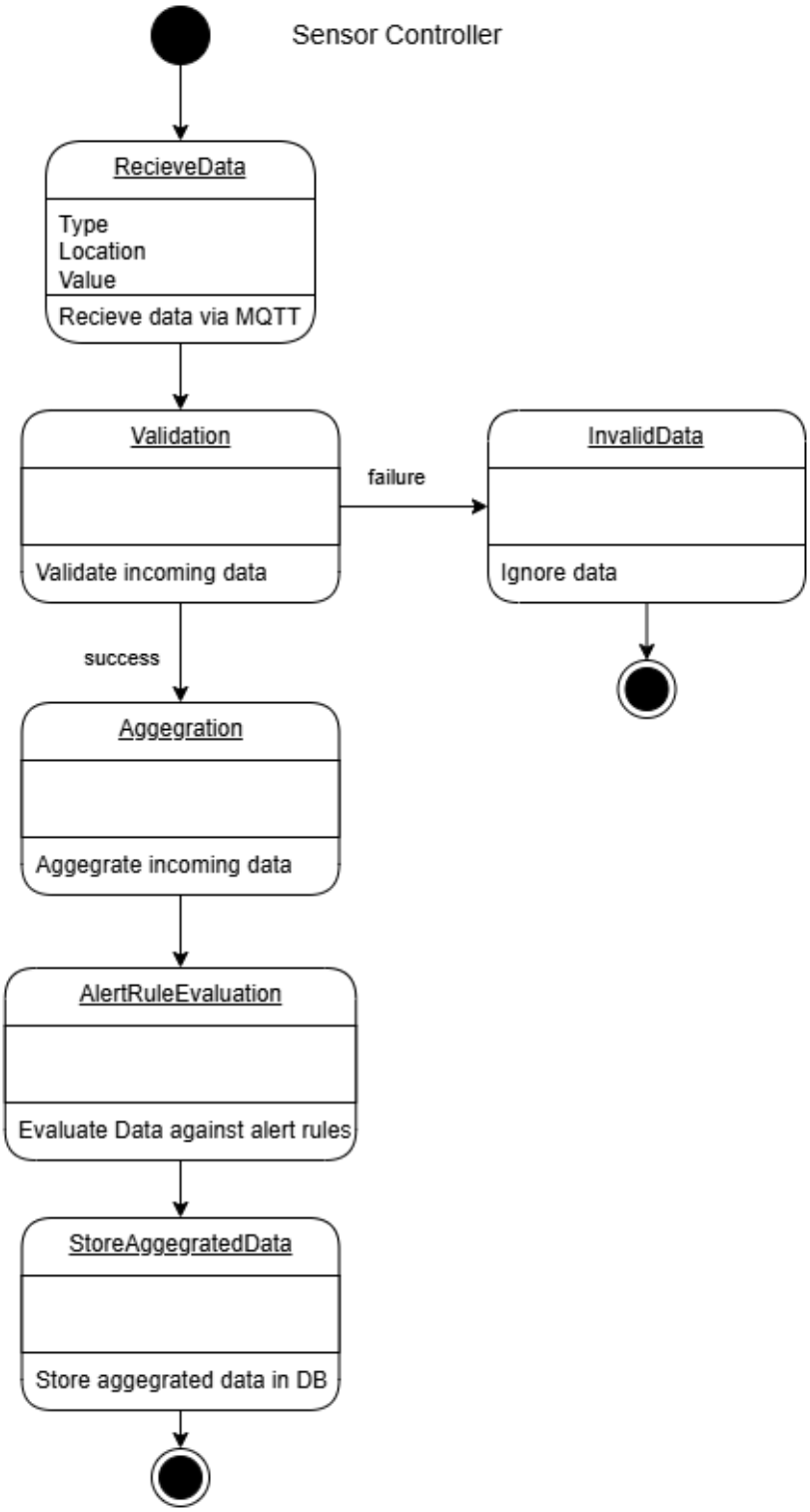

Figure 2 Sensor Controller State Diagram

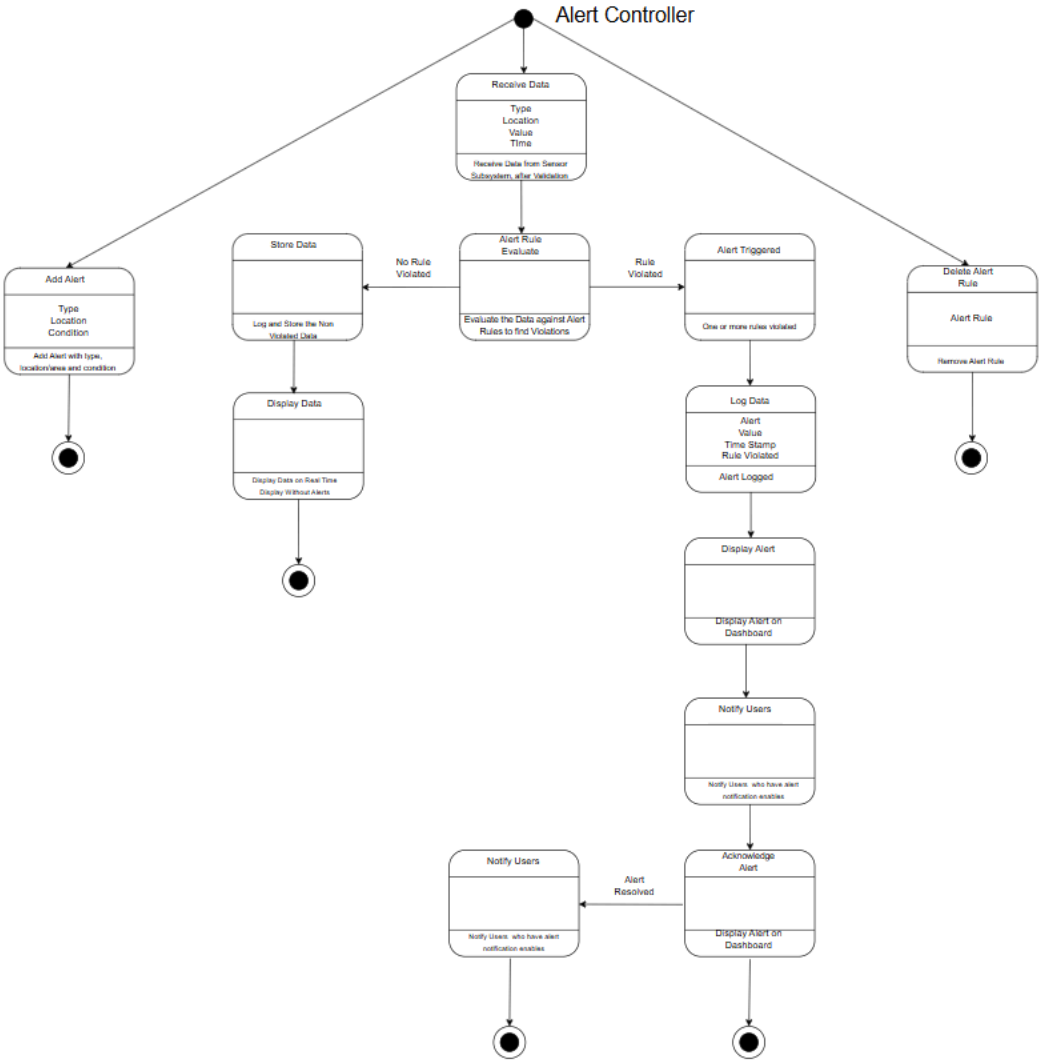

Figure 3 Alert Controller State Diagram

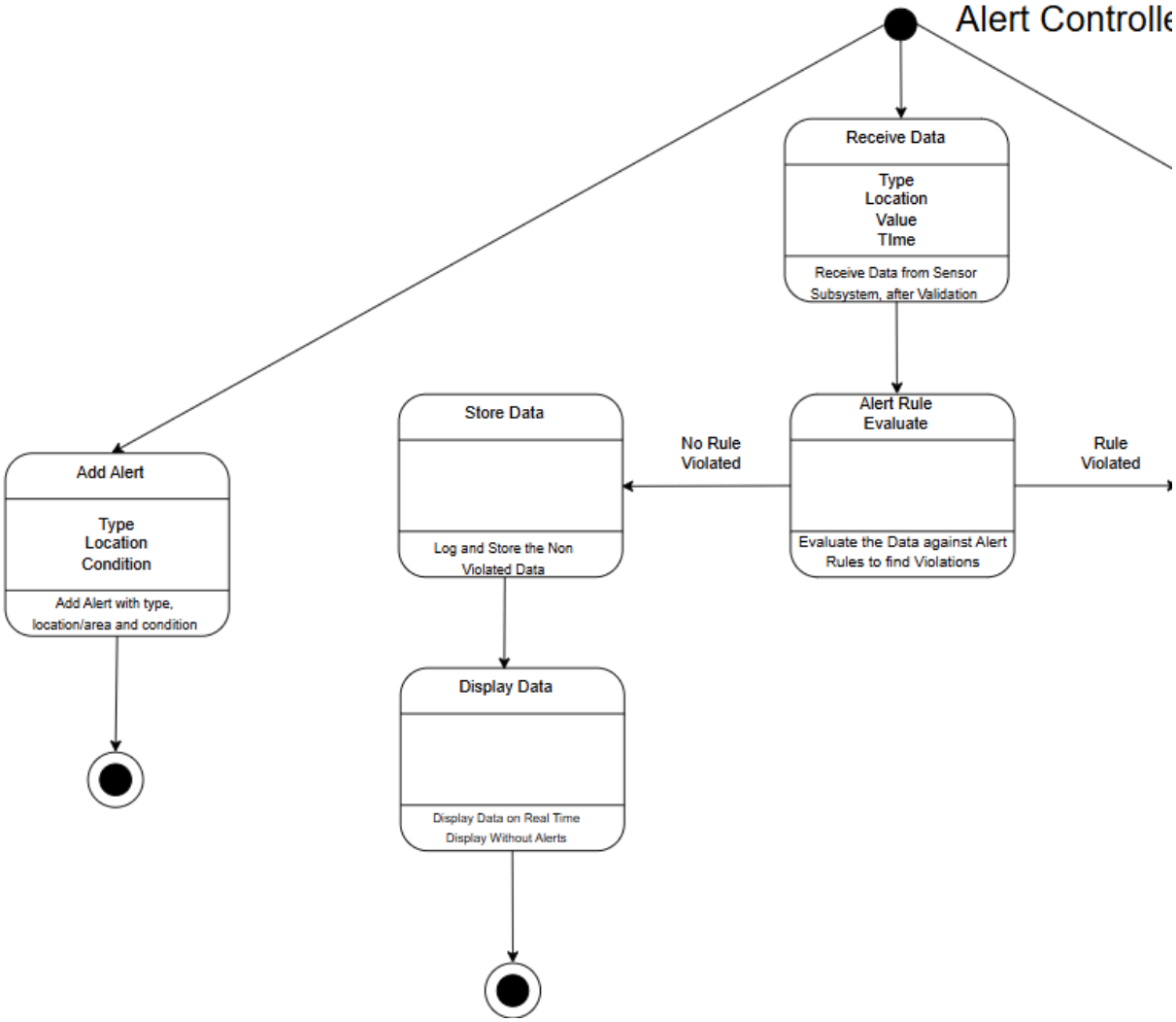

Figure 3.1 Alert Controller State Diagram 1

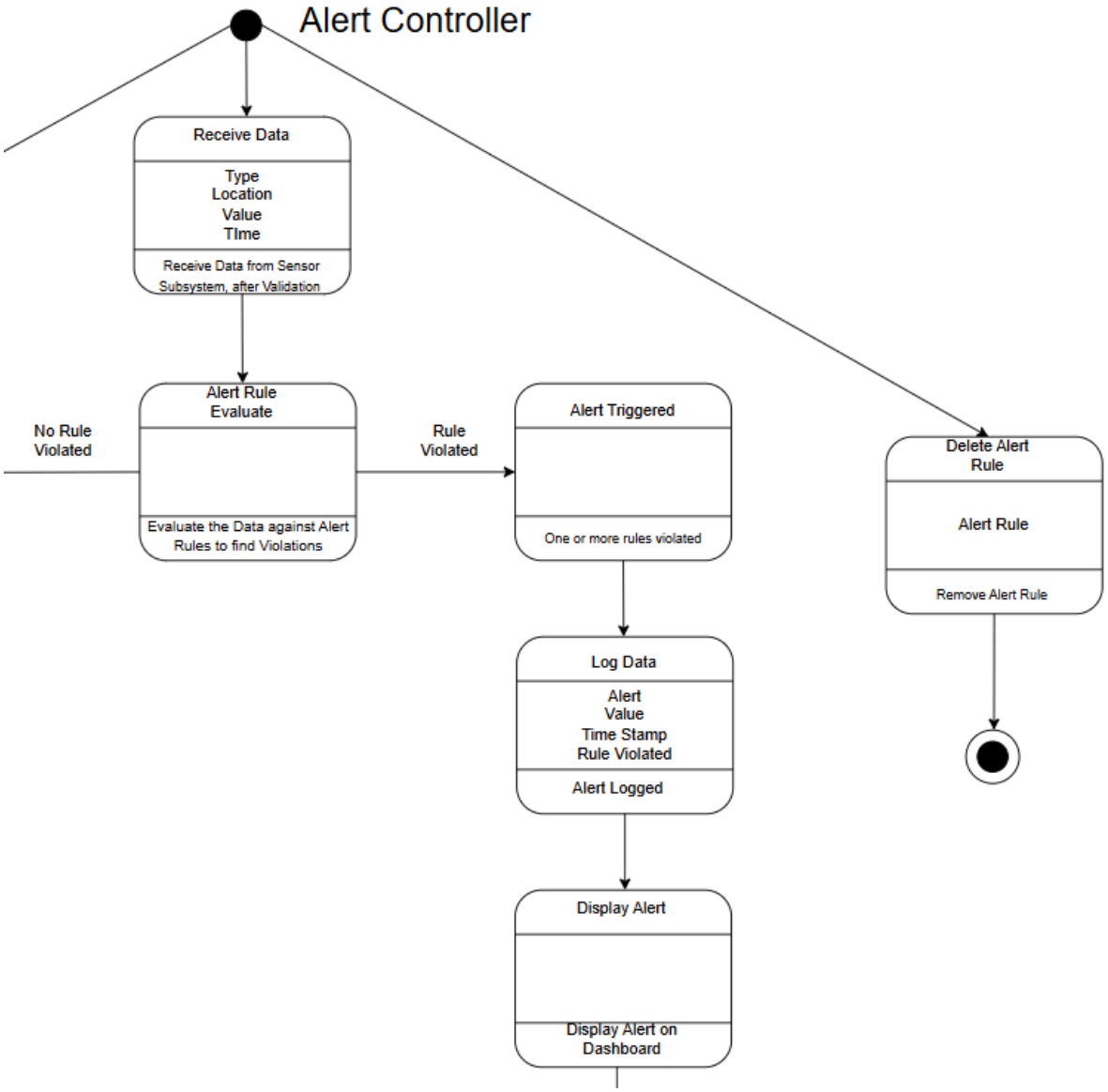

Figure 3.2 Alert Controller State Diagram 2

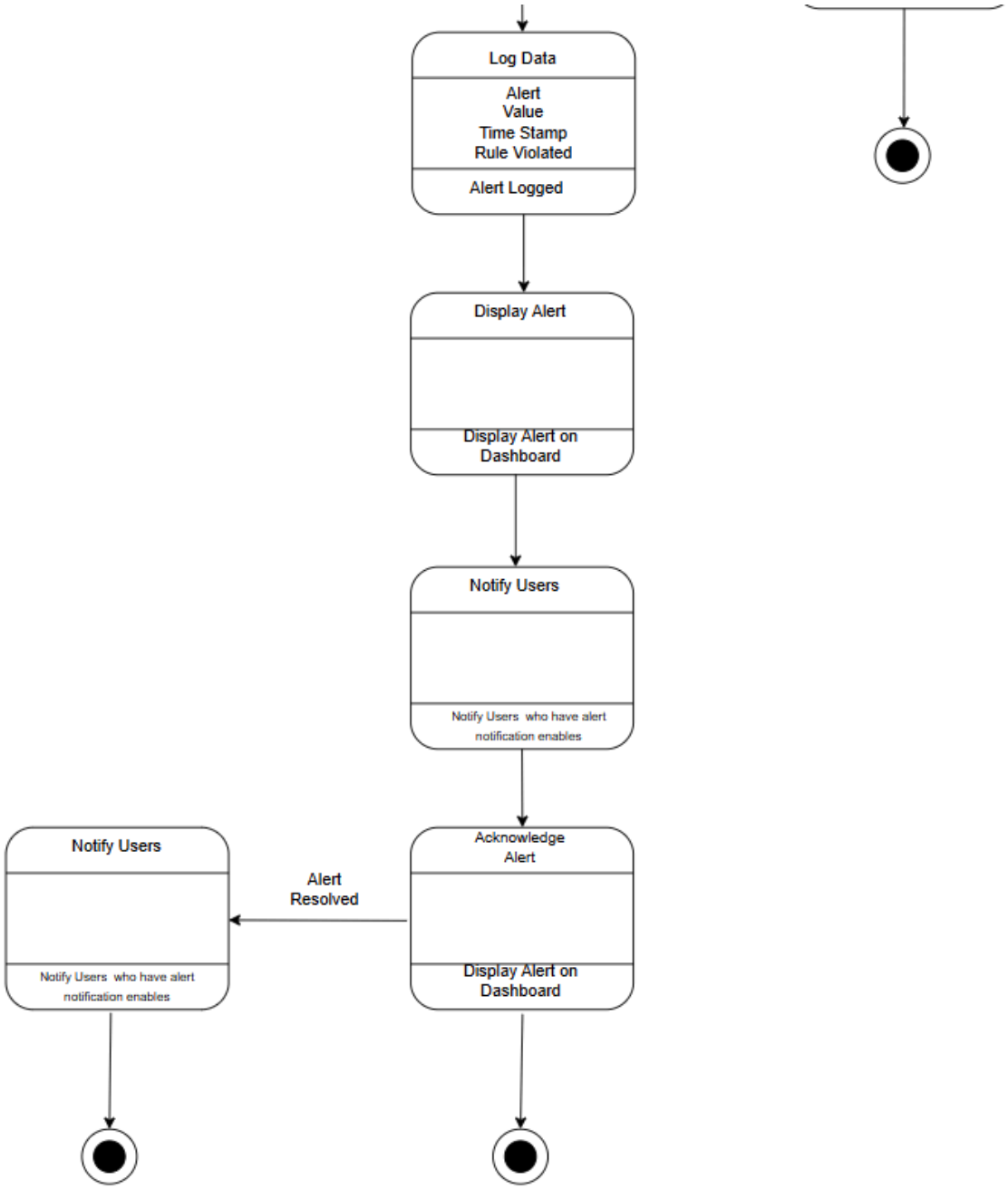

Figure 3.3 Alert Controller State Diagram 3

[3] Sequence Diagrams

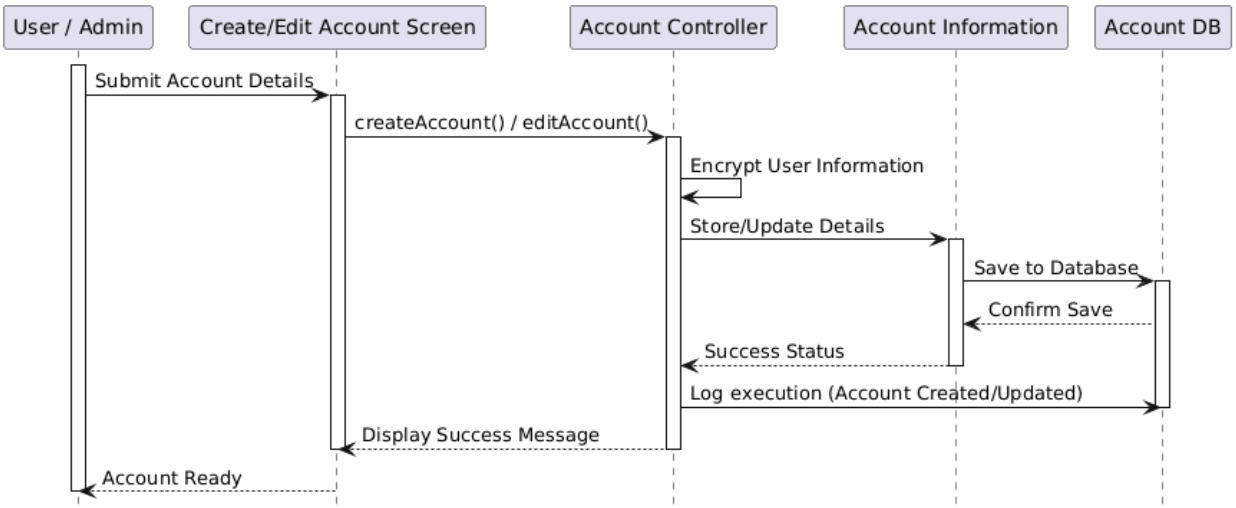

Figure 4. Account Creation & Information Update Sequence Diagram

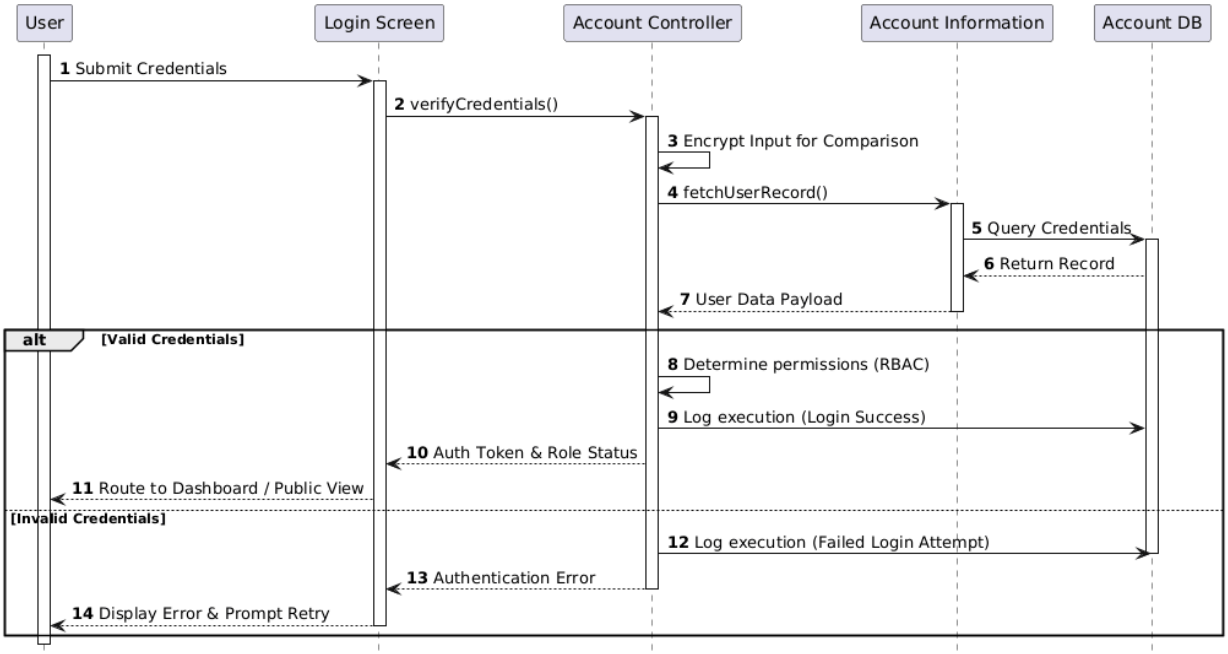

Figure 5. User Login & Authentication

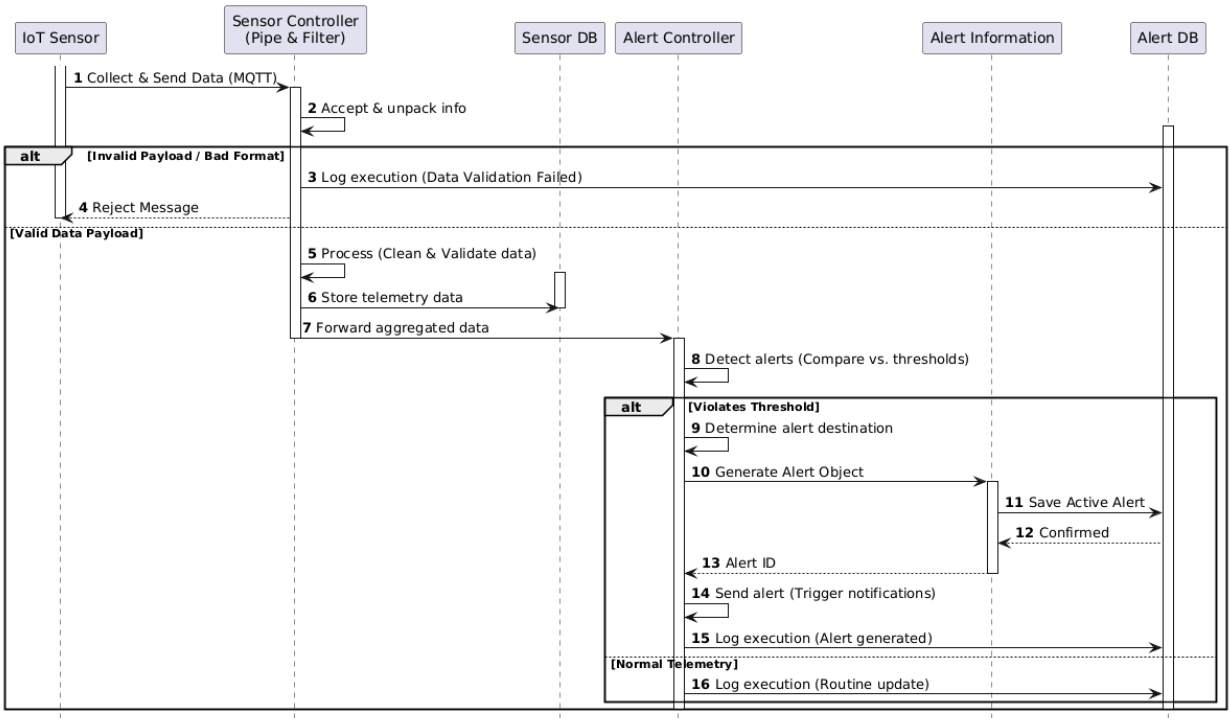

Figure 6. Sensor Telemetry & Auto-Alerting Pipeline

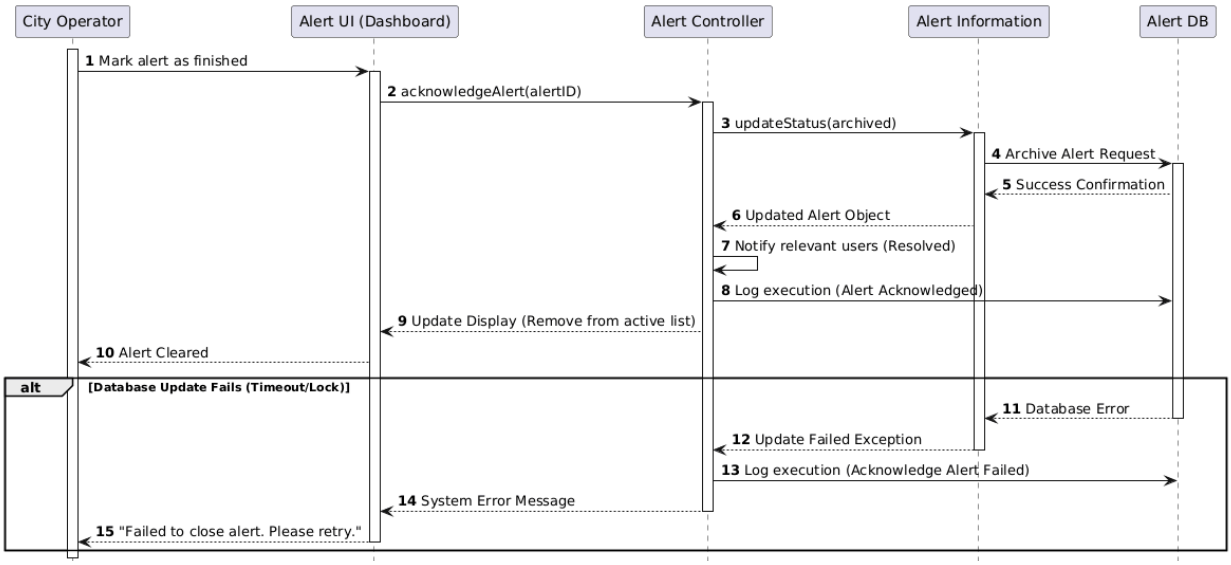

Figure 7. City Operator Dashboard & Alert Management

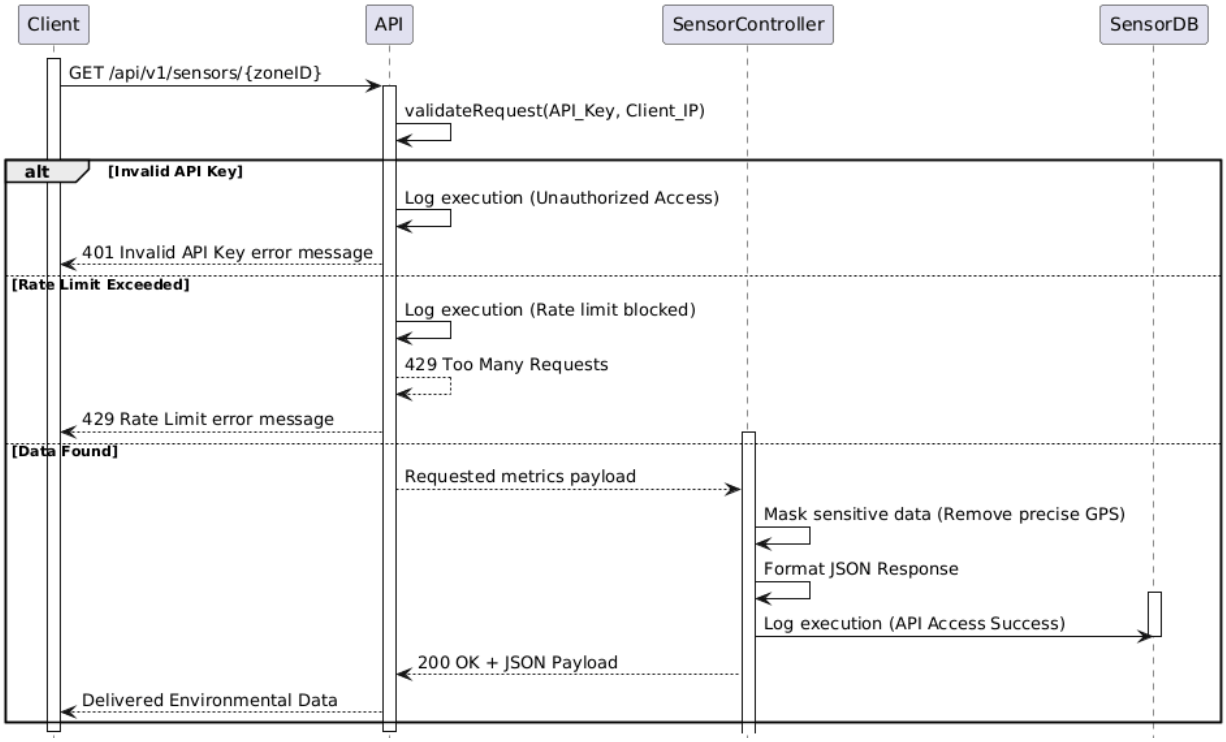

Figure 8. Public API

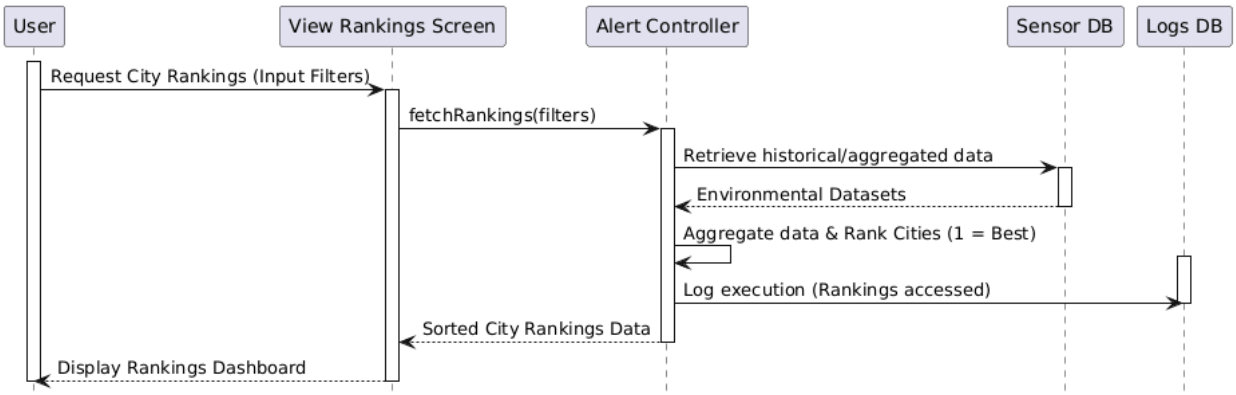

Figure 9. Generate City Rankings (Innovative Feature)

[4] Detailed Class Diagram

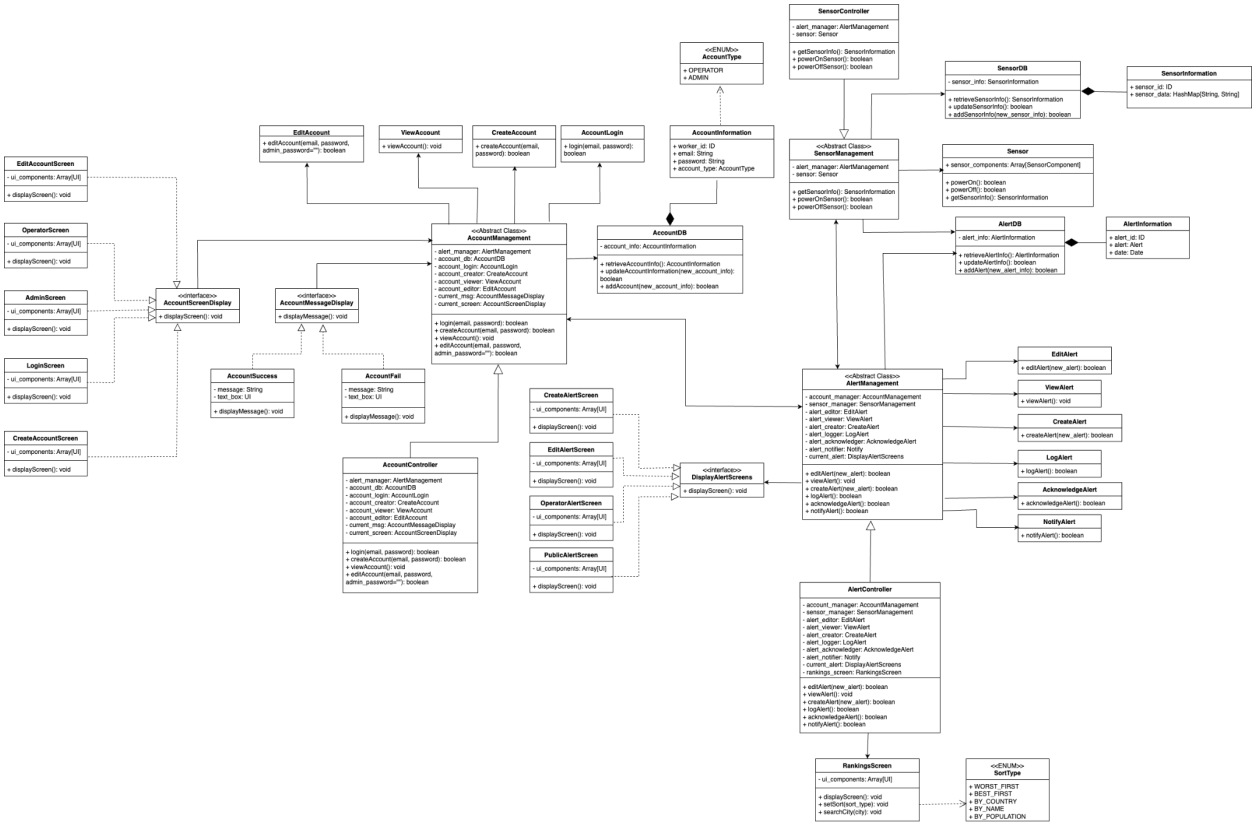

Figure 10. Detailed Class Diagram

[A] Division of Labour

Heathfield, Aidan:

- Helped a little bit with Account Controller state diagram in section 2
- 4

McEachern, Connor:

- 1.1
- 2 Account controller and Sensor controller

Saumy, Patel:

- **Section 2: Alert Controller**

Zafar, Ahmed:

-

- 1.2
- 1.3
- Section 3